

FROM SINGLE NODE TO DISTRIBUTED FOUNDATIONS

Production Infrastructure Engineering

Phase 21: How 3 weeks of data migration, containerization, and architectural decision records turned a single-node project into a truly distributed system

0 Mac Studio Jobs

Workstation out of workload rotation

3 Containerized Services

On Linux cluster nodes

1036 Tests

Zero regressions across 9 sessions

THE CHALLENGE: THE MAC STUDIO PIN

SINGLE POINT OF EVERYTHING

Every observability job in the cluster carried a placement constraint pinning it to one node — the development workstation.

Not for compute reasons — for *data* reasons. The SQLite database lived on Mac Studio's local filesystem. Writing to it required local access. Therefore

The system was shaped like a single-node project that happened to call itself distributed.

WHAT WORKED

- Zero-LLM analytical stack (5 phases)
- 8-tab Glass Box UI
- 980/980 tests passing
- Correlation engine production-ready

WHAT DIDN'T

- Single node for all writes
- Hard pin prevents rescheduling
- No compute portability
- Database disk = single point of data loss

"The single largest architectural debt item." — external portfolio feedback, Phase 20

THE VISION: THREE PORTABILITIES

Break the data pin. Break the code pin. Break the compute pin.

DATA

SQLite on one workstation
→ PostgreSQL on cluster
Network-accessible, multi-tenant
BRIN-indexed time-series

25,929 rows

Migrated with zero loss

CODE

sqlite3.connect() everywhere
→ single factory, two backends
%s placeholders everywhere
Python-side timestamp cutoffs

7 modules

Ported to backend-agnostic

COMPUTE

raw_exec + hardcoded paths
→ multi-arch Docker containers
ARM64 + AMD64 images
Shared base image pattern

3 services

Containerized

"All three, within three weeks, without regressing a single test."

THE THREE-WEEK STRATEGY

W 1

Foundation

1000th test milestone
PostgreSQL provisioning
Schema + data migration
Non-disruptive: SQLite still authoritative

1012 tests, 993 passed

W 2

Module Cutover

7 code paths → backend-agnostic
Python-side timestamp cutoffs
Dialect gates where needed
Cross-node writes proven

1014 tests, 995 passed

W 3

Containerization

Multi-arch OCI images
Private registry deployed
Mac Studio pin REMOVED
Alerting integration added

1036 tests, 1002 passed

Each week built on the last. Zero regressions across nine sessions.

WEEK 1: POSTGRESQL MIGRATION

rag/db.py

URL scheme routing
postgresql:// → Psycpg 3
sqlite:// → internal wrapper
137 lines, 14 tests

No SQL arithmetic

cutoff = int(time.time()) - s
Passed as bound parameter
Both backends identical SQL
Zero dialect branching

%s everywhere

Consumers write Psycpg SQL
Wrapper translates %s → ?
One dialect in consumer code
Translation in one place

Canonical migration

BRIN indexes on timestamps
TIMESTAMPTZ + BIGSERIAL
Idempotent via IF NOT EXISTS
Big-bang cutover strategy

KEY INSIGHT

The Python-side cutoff was the single most impactful line of the migration. When two SQL dialects diverge on computing a value, compute it in application code. Both dialects accept parameters identically.

WEEK 1 RESULTS

25,929 rows

Migrated with zero loss

13 new tests

Connection factory + routing

0 regressions

SQLite still authoritative

WEEK 2: MODULE CUTOVER + PIVOT

SESSION 6 BLOCKED

Session 6 was supposed to remove the Mac Studio pin. Pre-inspection killed the plan in 5 minutes.

The scraper used the raw_exec driver with hardcoded macOS paths. Removing the pin would let Nomad schedule the job on Linux — where the binary did not exist.

[NO] Symlinks + distributed virtualenvs

[NO] Bundled-interpreter artifact stanzas

[YES] Option C: Prove cross-node writes now, containerize in Week 3

LATENT BUG SURFACED

First redeploy crash-looped: ModuleNotFoundError: No module named 'rag'. Session 4 had added an import that never ran until Session 6's redeploy. The running alloc had cached the pre-Session-4 binary for 4 hours. Containerization eliminates this class of bug structurally.

CROSS-NODE WRITE PROOF

```
T0: count= 36036  
   m ax_ts= 1775945114  
T1: count= 36063  
   m ax_ts= 1775945159
```

```
DELTA rows    = 27  
DELTA seconds = 45  
ACROSS NODES = Mac Studio → lab03
```

Data plane portable.

Compute plane: staged for Week 3.

WEEK 3: DURABLE PLACEMENT POLICY

OPERATIONAL STATE

```
nom ad node eligibility -disable
```

Ephemeral — anyone can reverse it
Cluster-wide (all jobs)
Not version-controlled
Survives until next restart

Weeks later someone re-enables the node. Nomad places a Linux container on macOS. Runtime fails. The error does not point back to the placement decision.

ARCHITECTURAL INTENT

```
constraint { attribute "..." != "Mac Studio" }
```

Durable — committed to git
Job-specific (only what needs it)
Version-controlled audit trail
Survives cluster resets

Any change requires a commit. The constraint is part of the job definition. It cannot be silently reversed. Forced rescheduling still places correctly.

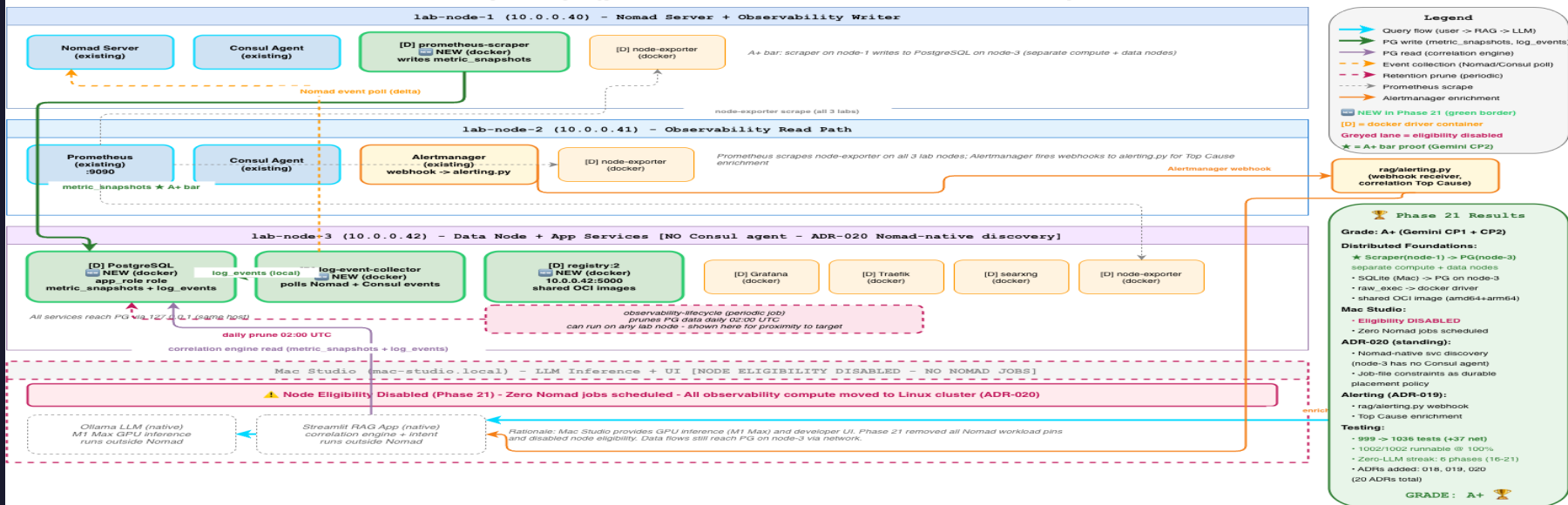
ADR-020 STANDING CONVENTION

Every Nomad job using a Linux-only Docker image carries the Mac Studio exclusion constraint. Three jobs established the pattern in Phase 21. Belt-and-suspenders: both layers (operational + architectural) together provide defense in depth.

PHASE 21 TOPOLOGY

Phase 21: Distributed Foundations - PostgreSQL + Containerization + ADR-020

Physical topology: 4 nodes, data flows, Mac Studio retired from Nomad compute



Phase 21 Complete (Grade: A+) | Date: April 12, 2026 | ADR-019 / ADR-019 / ADR-020 | Sessions: 9 technical + 2 portfolio chats

Achievement: SQLite on Mac Studio → PostgreSQL on node-3 | raw_exec → docker driver | Mac Studio retired from Nomad compute | Next: Phase 22 TBD

Compute on lab-node-1. Data on lab-node-3. Neither is the workstation.

PRODUCTION ENGINEERING

SCOPED ROLE

Initial migration: admin role (velocity)
Phase close: application role (CRUD only)
Credential stored as orchestrator variable
Cleartext never in committed files

MULTI-ARCH OCI

registry:2 on cluster data node
ARM64 + AMD64 images
docker buildx with push
force_pull = true on every deploy

WORKFLOW v1.7

Pre-restart: enumerate dependencies
Verify scheduling constraints
Born from a Docker restart incident
Codified in CHAT-SESSION-WORKFLOW.md

6 PHASES ZERO-LLM

Analytical stack fully deterministic
Pattern matching + closed-form math
Predictive engine: P95 ~0.05ms
LLM only on final synthesis

Production engineering is about what is repeatable — not what is clever.

BY THE NUMBERS

Axis	Phase 20 End	Phase 21 End	Delta
Compute location	Mac Studio (raw_exec)	Linux cluster (docker)	[+] Cluster
Data location	SQLite on workstation	PostgreSQL on lab03	[+] Shared
DB role	SUPERUSER	Scoped CRUD-only	[+] Least-priv
Container registry	None	Private multi-arch	[+] In-cluster
Containerized services	0	3	[+] +3
Mac Studio Nomad jobs	2	0	[+] Zero
Tests collected	999	1036	[+] +37
Tests passing	980	1002	[+] +22
Zero-LLM streak	5 phases	6 phases	[+] +1

0

Mac Studio Jobs

1002 / 1002

Runnable Tests @ 100%

6

Consecutive Zero-LLM Phases

Zero regressions across nine sessions. Every phase delivered tests.

ALERTING: PASSIVE → ACTIVE

PASSIVE OBSERVER

Phases 18-20 built the zero-LLM analytical stack. The system could answer 'why is this metric spiking?' in the UI with a full Top Cause reasoning trace.

But the reasoning stayed inside the UI.

An operator receiving an alert at 3am saw only the alert name and the raw value — same payload the monitoring system would emit without this project existing.

ACTIVE PARTICIPANT

enrich_alert(payload, db_url) — pure function.

Extracts metric + node from the alert. Calls the correlation engine for Top Cause. Appends annotation with entity, event type, score, inflection.

```
top_cause: correlation_candidate:  
  nom ad_allocation_b319419f  
  { type= healthcheck_fail,  
    score= 0.87,  
    inflection= step_change}
```

ADR-019 — LOGIC-ONLY SCOPE

24 tests, 6 documented failure modes, all returning the alert unchanged. The enrichment function cannot break an alert — only add value or fail silently. HTTP wrapper is a Day 2 task (external feedback: "logic-only is sufficient").

KEY ENGINEERING PRINCIPLES

Six lessons encoded from nine sessions.

DO NOT CONSOLIDATE

Week 1 checkpoint skipped cost half a session of rework. Checkpoints are course corrections, not status reports.

COMPUTE IN THE HOST LANGUAGE

When dialects diverge on computing a value, compute in Python. Pass as parameter. Dialect surface area drops to zero.

CHECK BEFORE PROVISIONING

"Provision X" is always suspect. One command to check if X already exists. Missing it can be catastrophic for unrelated services.

FORCED RESCHEDULING = AUDIT

Any event that forces re-placement is a cluster-wide constraint validation pass. Hidden gaps surface when allocs are evicted.

INTENT LIVES IN GIT

Two mechanisms, different purposes. Architectural intent must be durable. Ephemeral state is a time bomb.

NEVER BLOCK A SIGNAL

Any layer between a critical signal and its recipient should default to pass-through. Fail open for alerting, not fail closed.

Benchmark → Experiment → Validate → Pivot. Nine sessions. Zero regressions.

THE TRANSFORMATION

BEFORE PHASE 21

- Mac Studio ran everything
- SQLite on local disk
- raw_exec with hardcoded paths
- SUPERUSER database role
- Service blocks that silently failed
- 17 ADRs, no placement policy

"The system was shaped like a single-node project that happened to call itself distributed."

AFTER PHASE 21

- Three Linux nodes carry all workloads
- PostgreSQL, network-accessible, multi-tenant
- Docker driver, multi-arch, containerized
- Scoped CRUD-only role
- Native orchestrator service discovery
- 20 ADRs, durable placement policy (ADR-020)

"The shape finally matches the claim."

"True Computational Portability." — external validation, Gemini CP2

THE IMPACT

0

Mac Studio Jobs

A+

Gemini Grade

6

Zero-LLM Phases

Phase 21 is the first phase whose portfolio story is not about intelligence — it is about operational maturity. Database migration with zero downtime. Multi-arch containerization. Least-privilege IAM. Distributed compute. Architectural decision records capturing both planned decisions and unplanned insights. The methodology held across nine sessions: hypothesis-driven planning, weekly external validation, zero-regression standard, two-version documentation.

METHODOLOGY

Session 0 with Gemini validation
Weekly checkpoint cadence (v1.7)
Infrastructure Blast Radius Checklist
Two-version ADR strategy (ADR-018, 019, 020)

PHASE 22 — SELF-HEALING AGENTS

Top Cause → automated remediation
conversation_memory.db migration decision
Consul on lab03 evaluation
Tab 7 E2E test harness